University of Pisa

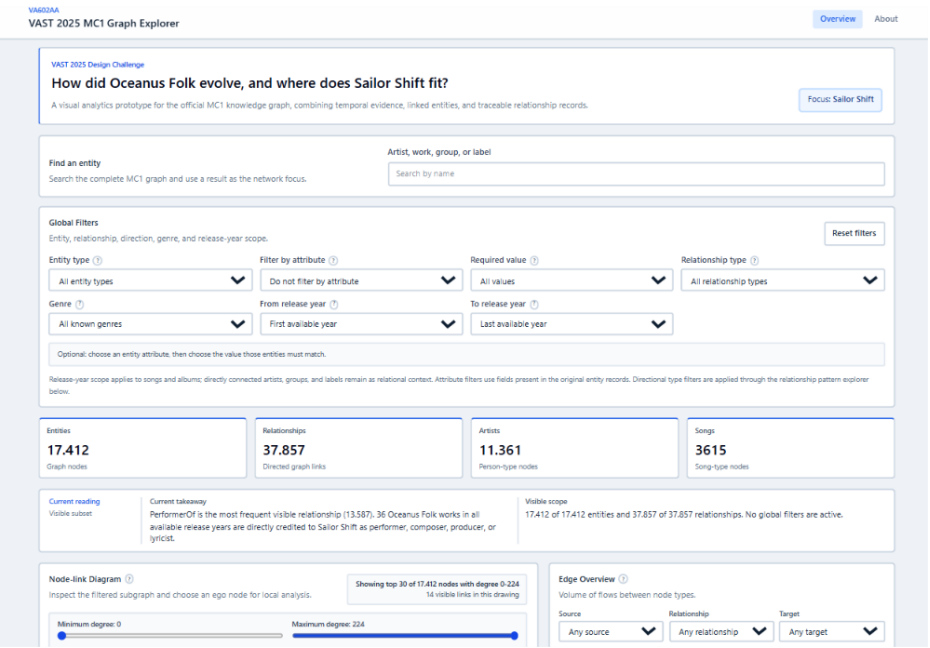
Department of Computer Science

Reading a Music Knowledge Graph

A Visual Analytics Prototype for VAST 2025 MC1

Course: Visual Analytics 602AA
Student: Alissia Biliotti
Project: VAST Challenge 2025, Mini-Challenge 1
Repository: `github.com/alissiabi/VASTKnowledgeGraphVisualization_AB`

Project summary. This project proposes a coordinated visual analytics workflow for exploring the VAST 2025 MC1 knowledge graph, moving from relationship overview to local graph inspection and evidence verification.



Academic Year 2025/2026

Abstract

This report presents a visual analytics prototype for exploring the VAST 2025 Mini-Challenge 1 music knowledge graph. The dataset contains heterogeneous entities and typed directed relationships, making it difficult to understand through either a table or a complete node-link diagram. The prototype addresses this problem by combining relationship summaries, filtered graph views, ego-network inspection, temporal context, path discovery, comparison tools, and evidence tables. Its main contribution is an analytical workflow that helps the user move from a global pattern to a local explanation and finally to the underlying data records.

Contents

1 Introduction

Knowledge graphs are useful because they represent information as entities connected by typed relationships. This structure is expressive: it can show people, songs, albums, record labels, groups, genres, and the relationships among them in a single model. At the same time, this expressiveness creates a visual analytics challenge. A large node-link diagram can easily become unreadable, while a table of relationship records hides the topology that makes the graph meaningful.

The VAST 2025 Mini-Challenge 1 dataset is a good example of this problem. It contains a heterogeneous music knowledge graph with different entity types and directed relationships. The analyst may need to understand which kinds of relationships dominate the graph, whether an entity acts as a local hub, how two entities are connected, or whether a visible pattern is supported by the original records. These tasks require both visual overview and detailed verification.

The prototype developed for this project is designed around three analytical needs:

- **Relationship understanding**: identify which combinations of source type, relationship type, and target type are most relevant in the current graph context.
- **Local inspection**: move from a global pattern to the neighborhood of a selected entity.
- **Evidence checking**: connect a visual interpretation to the original relationship records and available metadata.

For this reason, the project is not structured as a static dashboard of independent charts. It is a coordinated workspace where each view supports a step of the same analytical reasoning process.

1.1 Dataset

The prototype uses the VAST 2025 MC1 music knowledge graph. The graph contains music-related entities such as people, songs, albums, record labels, and musical groups. Links represent typed relationships, including performance, composition, production, recording, distribution, membership, sampling, interpolation, lyrical references, and style-related information.

Dataset	Nodes	Links	Description
VAST 2025 MC1	17,412	37,857	Directed heterogeneous music knowledge graph

Table 1: Structural profile of the graph used in the prototype.

The direction of relationships is preserved throughout the interface. This is important because a directed edge is not simply a connection between two nodes: it also expresses a role. For example, a relation from a person to a song has a different meaning from a relation from a label to an album. Preserving direction helps avoid ambiguous interpretations.

Before visualization, the data loader normalizes identifiers, resolves source and target nodes, extracts readable labels, and preserves raw node and link properties. This preprocessing step allows the interface to show both simplified summaries and detailed records when needed.

2 Design Rationale

The main design decision is to avoid presenting the knowledge graph as one large drawing. A complete node-link visualization of the whole dataset would be visually dense and difficult to interpret. Instead, the prototype separates the analysis into complementary views: some views

summarize the graph, some expose a filtered subset, and others focus on a selected entity or relationship.

The system follows four design principles.

1. **Start from structure, not from isolated records.** The user first receives a summary of the graph and its relationship patterns, then moves toward details.
2. **Keep graph drawings bounded.** Node-link views are useful only when the number of displayed nodes and edges is controlled.
3. **Preserve direction and type.** Source type, target type, and relationship type are treated as core information, not as optional metadata.
4. **Make interpretation verifiable.** A visual pattern should be connected to entity details, relationship details, path information, or observed records.

These principles shape the interface as a workflow. The analyst can begin from an overview, apply filters, inspect a graph subset, select a node, explore its neighborhood, and finally check the evidence behind the pattern.

3 System Overview

The prototype is implemented as a Vue single-page application. Vue manages the application state and component structure, while D3 is used for visual encodings that require geometric layout or custom drawing. The graph is loaded from a local JSON file, normalized in the browser, and indexed to support repeated interactions.

The interface is organized around a main dashboard. The dashboard includes global controls, overview summaries, graph-oriented panels, and evidence-oriented panels. Figure 1 gives an overview of the interface and shows how the main components are arranged in the same workspace.

The most important components are:

- an edge overview for aggregate relationship patterns;
- a filtered node-link diagram for the current graph subset;
- an ego-network view for the selected entity;
- a timeline for temporal context based on available release years;
- a degree distribution view for connectivity structure;
- a path discovery view for connections between entities;
- detail panels for selected nodes and links;
- evidence tables for inspecting observed relationship records;
- comparison tools for selected artists.

The system is read-only: it supports exploration and verification without altering the original graph records.

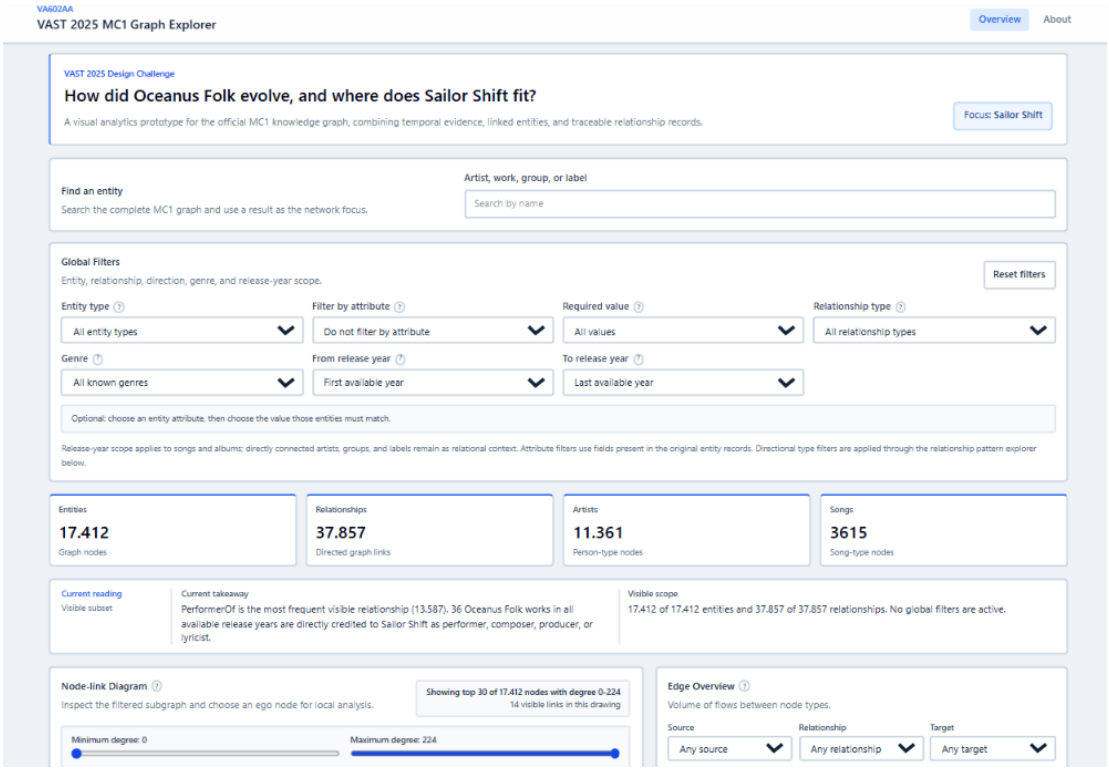


Figure 1: Overview of the prototype dashboard. The interface combines graph summaries, relationship-oriented views, local inspection, and supporting evidence panels in a single analytical workspace.

3.1 Shared State and Coordination

The dashboard is coordinated through a small set of shared states. This is important because visual analytics systems can become confusing if each panel applies its own filters or interprets selections independently. In this prototype, the main dashboard component stores the current filters, selected entity, selected relationship, network settings, and path target. These states are passed explicitly to the views that need them.

Shared state	Function in the interface
Filters	Define the visible subset used by summaries, relationship overview, and graph views.
Selected entity	Drives the entity detail panel, ego network, path search, and comparison tools.
Selected relationship	Opens the source, target, direction, relationship type, and available properties of a link.
Network settings	Control limits such as ego depth, relationship focus, and maximum number of nodes.
Path target	Defines the destination entity for connection-path exploration.

Table 2: Main shared states used to coordinate the dashboard.

This coordination allows the user to move between levels of analysis without restarting the exploration. For example, selecting an entity in the graph can update the local neighborhood, detail panel, and comparison tools. Selecting a relationship can move the analysis from a visual edge to its observed properties.

4 Analytical Components

Each component is designed to answer a specific question. This is important because a single visualization cannot answer all questions about a large heterogeneous graph. Table 3 summarizes the main analytical role of each view.

Component	Analytical question	Main information shown
Graph summary	How large is the current graph context?	Node count, link count, active filters, selected elements.
Edge overview	Which source–relationship–target patterns dominate?	Aggregated relationship flows and counts.
Node-link diagram	What does the filtered graph subset look like?	Visible nodes, links, types, direction, and connectivity.
Ego network	What surrounds the selected entity?	Local neighborhood, direct and indirect connections, relationship directions.
Timeline	How are selected works distributed over time?	Release-year distribution for songs and albums where available.
Degree distribution	Is connectivity concentrated around few entities?	Degree frequencies and connectivity imbalance.
Path discovery	How can two entities be connected?	Sequence of intermediate nodes and relationships.
Evidence table	Which records support the current interpretation?	Original relationship rows and endpoint metadata.
Evidence audit	Which records may require further inspection?	Missing values, duplicate-like patterns, chronology signals, and quality cues.
Artist comparison	How do two selected entities differ or overlap?	Profiles based on graph relationships and available metadata.

Table 3: Dashboard components and their analytical role.

4.1 Relationship Overview

The relationship overview is the first step in the analysis. Instead of showing every edge separately, it aggregates relationships by source type, relationship type, and target type. This makes the graph readable at a higher level.

This view is useful because heterogeneous knowledge graphs are not only about which nodes are connected. They are also about which kinds of entities interact through which kinds of relationships. A large number of links between people and songs has a different analytical meaning from a large number of links between labels and albums.

The edge overview therefore helps the analyst identify a meaningful slice before entering a more detailed graph view. It supports questions such as:

- Which relationship types dominate the dataset?
- Are some source–target combinations more frequent than expected?
- Are there relationship directions that deserve closer inspection?

Figure 2 shows the part of the interface dedicated to this transition from aggregate relationship patterns to graph-level inspection.

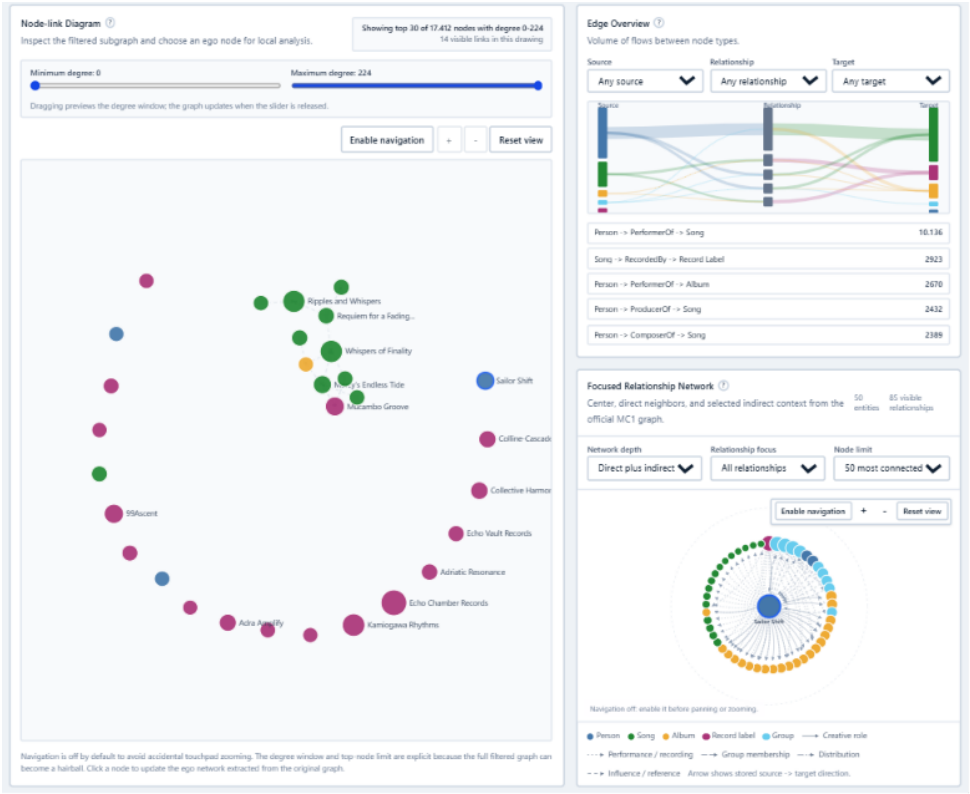


Figure 2: Relationship-oriented workflow. Aggregate relationship patterns guide the analyst toward a filtered graph subset, while node selection opens local inspection through the ego network and detail panels.

4.2 Filtered Node-Link Diagram

The node-link diagram shows the graph induced by the active filters. It is not intended to represent the entire MC1 graph at once. Instead, it provides a bounded topological view of the current analytical context.

Nodes are visually distinguished by entity type, while links preserve relationship type and direction. Node size can reflect local connectivity, helping the user notice hubs or highly connected entities inside the current subset. Since graph drawings become difficult to read when too many elements are shown, the diagram includes explicit limits and controls.

This component is useful after the user has already narrowed the graph through filters or relationship selection. Its main role is to show how the selected subset is connected.

4.3 Ego Network

The ego network focuses on the neighborhood of the selected entity. It is separated from the filtered node-link diagram because the two views answer different questions. The node-link diagram asks what the current filtered subset looks like. The ego network asks what is connected to this specific entity.

This distinction matters. If a user selects an artist after applying strong filters, the filtered graph may show only part of the artist's context. The ego network makes the local structure explicit and helps avoid confusing a filtered subset with the complete neighborhood of the entity.

The ego-network view supports questions such as:

- Which entities are directly connected to the selected node?
- Are incoming and outgoing relationships balanced?
- Which relationship types dominate the local context?
- Does the selected entity connect to different areas of the graph?

4.4 Temporal Context

Temporal analysis is based mainly on release-year information associated with songs and albums. The timeline does not claim that every relationship has a date. Instead, it uses available temporal metadata to provide context for works connected to the current selection or filter.

This distinction is important. A relationship record may say that an artist performed a song, but the relationship itself may not have an independent timestamp. The system therefore treats temporal views as contextual evidence, not as a complete history of all graph events.

4.5 Path Discovery

Path discovery helps the analyst understand how two entities are connected through the graph. A path is not automatically an explanation, but it can reveal intermediate entities or relationship chains that deserve inspection.

For example, two artists may be connected through a shared song, a label, a group, or a chain of production and recording relationships. The path view makes these intermediate steps explicit. This supports interpretation because the user can see not only that two entities are connected, but also how the connection is constructed.

4.6 Evidence-Oriented Views

The evidence table and evidence audit are included to prevent the analysis from remaining purely visual. Visual patterns can suggest hypotheses, but they should be checked against the underlying records.

The evidence table exposes the observed relationship rows and relevant endpoint information. The evidence audit highlights records that may deserve additional attention, such as missing years, repeated relationship structures, or chronology-related signals. These signals are not treated as automatic errors. They are prompts for inspection.

In the prototype, these views are deliberately text- and record-oriented. They are less useful as screenshots because their role is not to create a compact visual pattern, but to let the analyst verify exact rows, endpoint metadata, and possible data-quality signals.

5 Example Analytical Workflow

A typical analysis begins with the graph summary and relationship overview. The analyst first looks at the dominant relationship patterns and chooses a relationship family or source–target combination to inspect. This step reduces the complexity of the graph before any node-link diagram is interpreted.

After filtering, the node-link diagram shows the current subset. The analyst can identify visible hubs, isolated groups, or repeated structural patterns. Selecting an entity then updates the local inspection views. The ego network shows the selected entity's neighborhood, while the detail panel exposes its metadata and relationships.

If the analyst wants to understand a connection between two entities, the path discovery view provides a sequence of intermediate nodes and relationships. If the question involves time, the timeline provides release-year context for songs or albums. Finally, the evidence table allows the analyst to inspect the records supporting the selected pattern.

The intended reasoning chain is therefore:

$$\textit{overview} \rightarrow \textit{filter} \rightarrow \textit{topology} \rightarrow \textit{local context} \rightarrow \textit{evidence}$$

This workflow is the main contribution of the project. The system does not simply show many charts; it organizes the analysis so that each step can be traced back to data.

6 Analytical Use on MC1

The MC1 graph contains heterogeneous and directed relationships, so the prototype is especially useful for inspecting direction, local context, and record-level consistency.

One relevant use case is relationship-direction inspection. In a music knowledge graph, the direction of a relationship is part of its meaning. If a relationship type is expected to connect works to labels, but some records appear in the opposite direction, the analyst needs a way to notice and verify this. The edge overview can reveal unusual source–target combinations, while the evidence table can be used to inspect the corresponding records.

A second use case is artist-centered exploration. Starting from a selected artist, the ego network can reveal songs, albums, groups, labels, and other related entities. The user can then move from the local neighborhood to path discovery or comparison tools. This supports questions about whether two artists are connected through shared works, shared labels, or indirect graph paths.

A third use case is temporal contextualization. When release years are available, the timeline can show whether a selected genre, artist context, or work subset is concentrated in a specific period or distributed over time. This does not replace detailed musicological analysis, but it helps the analyst formulate better questions about the data.

7 Implementation

The implementation is based on Vue, JavaScript, D3, SVG, and CSS. The code flow is organized around four steps:

1. loading and normalizing the graph;
2. storing the shared dashboard state;
3. computing filtered subsets and derived summaries;
4. rendering reusable visual components.

D3 scales are used to map values to position, length, size, and color. Vue propagates state changes across components so that one interaction can update all relevant views. This separation keeps the interface modular while preserving cross-view coordination.

8 Validation

The implementation was checked through code-level and build-level validation. The following commands were used during development:

- `npm run lint`
- `npm test`
- `npm run build`

These checks are useful but limited. Linting and build success confirm that the application is technically consistent, while tests help verify selected functions and transformations. They do not prove that every visual interpretation is correct. For this reason, the report distinguishes between implementation validation and analytical interpretation.

The project was also checked conceptually by comparing the behavior of the views with the intended analytical workflow. The central question was whether a user can move from a visual pattern to a more detailed inspection without losing context. The shared state model, detail panels, and evidence-oriented views were designed to support this requirement.

9 Scope and Limitations

The prototype is designed for exploration, not for automatic discovery. It helps the analyst inspect patterns and verify them, but it does not decide which patterns are important or correct.

Graph drawings are intentionally bounded by filters and node limits. This improves readability, although it prevents the user from seeing the full topology as a single drawing. This is an intentional design choice: a complete node-link diagram of the MC1 graph would not be analytically useful.

Temporal views depend on available release-year metadata and should be read as contextual evidence, not as a complete timestamped history of relationships. Similarly, evidence-audit signals such as missing values, repeated structures, or chronology cues indicate records worth checking, but they do not prove errors by themselves.

Finally, the interface remains a prototype. It demonstrates a visual analytics workflow for MC1, but it is not a production system for general-purpose knowledge graph management.

10 Conclusion

This project presents a visual analytics prototype for reading the VAST 2025 MC1 music knowledge graph. The system addresses the difficulty of exploring a large, heterogeneous, directed graph by separating the analysis into coordinated steps: relationship overview, filtered topology, local entity inspection, path discovery, temporal context, and evidence checking.

Its main contribution is the workflow: visual patterns can be refined through interaction and verified through the underlying records. Within its prototype scope, the system makes the graph more readable without reducing it to either a static table or an unreadable full-graph drawing.

Acknowledgement

I used Codex as an implementation assistant for parts of the Vue/D3 prototype, mainly for code organization, debugging, testing, and refinement. The analytical goals, design rationale, final interpretation, and report decisions are my own.